

1. Java Date and Time

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    /*
     * Complete the 'findDay' function below.
     *
     * The function is expected to return a STRING.
     * The function accepts following parameters:
     * 1. INTEGER month
     * 2. INTEGER day
     * 3. INTEGER year
     */

    public static String findDay(int month, int day, int year) {
        Calendar c = new GregorianCalendar(year, month-1, day);
        return c.getDisplayName(Calendar.DAY_OF_WEEK, Calendar.LONG,
Locale.getDefault()).toUpperCase();
    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
```

```

        BufferedWriter bufferedWriter = new BufferedWriter(new
        FileWriter(System.getenv("OUTPUT_PATH")));

        String[] firstMultipleInput =
        bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");

        int month = Integer.parseInt(firstMultipleInput[0]);

        int day = Integer.parseInt(firstMultipleInput[1]);

        int year = Integer.parseInt(firstMultipleInput[2]);

        String res = Result.findDay(month, day, year);

        bufferedWriter.write(res);
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

2.Java Priority Queue

```

import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;
import java.util.PriorityQueue;

class Student implements Comparable<Student> {
    private int id;
    private String name;
    private double cgpa;

    public Student(int id, String name, double cgpa) {
        this.id = id;
        this.name = name;
        this.cgpa = cgpa;
    }
}

```

```

    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public double getCGPA() {
        return cgpa;
    }

    public int compareTo(Student cStudent) {
        if (cgpa != cStudent.getCGPA()) {
            return Double.compare(cStudent.getCGPA(), cgpa);
        } else if (!name.equals(cStudent.getName())) {
            return name.compareTo(cStudent.getName());
        } else {
            return Integer.compare(id, cStudent.getId());
        }
    }
}

class Priorities {
    public List<Student> getStudents(List<String> events) {

        List<Student> servedStudents = new ArrayList<>();
        PriorityQueue<Student> queue = new PriorityQueue<Student>();

        for (String event : events) {
            String[] parsedEvent = event.split("\\s+");

            if ("ENTER".equals(parsedEvent[0])) {
                Student student = new Student(
                    Integer.parseInt(parsedEvent[3]), parsedEvent[1],
                    Double.parseDouble(parsedEvent[2]));
                queue.add(student);
            }
        }
    }
}

```

```

        } else if ("SERVED".equals(parsedEvent[0])) {
            queue.poll();
        }
    }
}

```

```

        while (!queue.isEmpty()) {
            servedStudents.add(queue.poll());
        }
    }
}

```

```

        return servedStudents;
    }
}

```

```

public class Solution {
    private final static Scanner scan = new Scanner(System.in);
    private final static Priorities priorities = new Priorities();

    public static void main(String[] args) {
        int totalEvents = Integer.parseInt(scan.nextLine());
        List<String> events = new ArrayList<>();

        while (totalEvents-- != 0) {
            String event = scan.nextLine();
            events.add(event);
        }

        List<Student> students = priorities.getStudents(events);

        if (students.isEmpty()) {
            System.out.println("EMPTY");
        } else {
            for (Student st: students) {
                System.out.println(st.getName());
            }
        }
    }
}

```

3.Java Comparator

```
public class Solution {
    public static void main(String[] args) {
        /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class
        should be named Solution. */
        try (BufferedReader bIn = new BufferedReader(new InputStreamReader(System.in)))
        {
            String line = bIn.readLine();
            List<Player> scoreList = new ArrayList<>();
            while ((line = bIn.readLine()) != null) {
                scoreList.add(new Player(line));
            }
            ScoreCompare cp = new ScoreCompare();
            scoreList.stream().sorted(cp::compare).forEach(elem ->
            System.out.println(elem.getName() + " " + elem.getScore()));
        } catch (Exception e) {
            //
        }
    }

    public static class Player {
        private String name;
        private int score;

        public Player(String text) {
            String[] strArray = text.split(" ");
            this.name = strArray[0];
            this.score = Integer.valueOf(strArray[1]).intValue();
        }

        public String getName() {
            return this.name;
        }

        public void setName(String name) {
            this.name = name;
        }
    }
}
```

```
        public int getScore() {  
            return this.score;  
        }  
  
        public void setName(int score) {  
            this.score = score;  
        }  
    }  
  
    public static class ScoreCompare implements Comparator<Player> {  
        @Override  
        public int compare(Player p1, Player p2) {  
            if (p1.getScore() < p2.getScore()) {  
                return 1;  
            } else if (p1.getScore() > p2.getScore()) {  
                return -1;  
            } else {  
                return p1.getName().compareTo(p2.getName());  
            }  
        }  
    }  
}
```